DATA INTEGRITY

What

Data Integrity ensures that information entered and processed through the Automated Network Request Management solution remains complete, accurate, consistent, and valid throughout the request lifecycle.

The solution uses mandatory-field validation, auto-population, UI Policies, Catalog Client Scripts, and approval-state checks to maintain the quality and consistency of request data.

The major data-integrity controls implemented are:

- Mandatory field validation
- Auto-population of relevant values
- Catalog variable-to-table mapping
- Approval-state validation
- Role-based access controls
- Full-cycle testing and validation

Why

Accurate request data is essential for successful network request fulfilment.

Incomplete or incorrect information can result in:

- Delayed request processing.
- Incorrect assignment.
- Failed approvals.
- Manual rework.
- Incorrect notifications.
- Inaccurate reporting.
- Compliance and audit issues.

Therefore, data-integrity controls are implemented at different stages of the request lifecycle to ensure that only valid and sufficiently complete information proceeds through the automation.

How

Mandatory Field Validation

What

Required fields are configured so that users cannot submit incomplete requests.

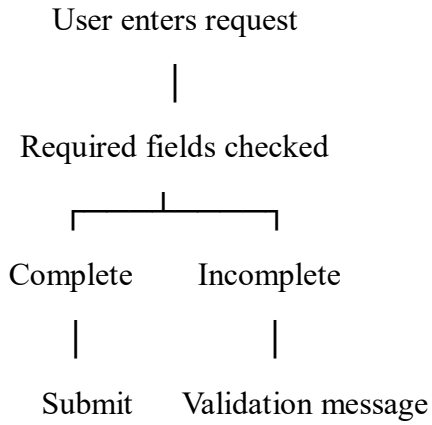
How

Mandatory validation can be enforced through:

- Catalog item variable configuration.
- Client-side validation/UI behavior.
- Server-side validation where applicable.

Fields that are essential for processing the Network Request are marked as **Mandatory**.

Example



Outcome

Incomplete requests are prevented from proceeding, reducing data-quality issues and manual corrections.

Auto-Population Logic

What

Relevant information is automatically populated based on the user's selection or the current user context.

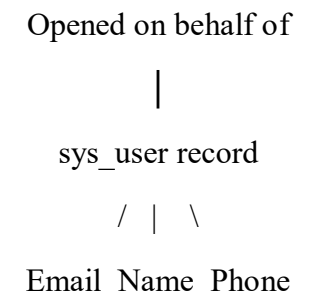
Examples include:

- Current user/requester information.
- User email.
- User name.
- Phone number.
- Other applicable contextual information.

How

Auto-population is implemented using the configured **Catalog Client Scripts, UI Policies, and variable relationships**.

For example:



This reduces the amount of information that users need to manually enter.

Outcome

Automatic population improves data accuracy and reduces repetitive data entry.

Data Handling

What

Information submitted through catalog variables is mapped to structured fields in the **Network Database** table.

How

The **Get Catalog Variables** action retrieves the information entered by the user.

The **Create Record** action then maps the retrieved values to the corresponding fields in the Network Database table.

Outcome

Request information is stored in a structured and consistent format, making it available for approval, fulfilment, reporting, and tracking.

Approval State Validation

What

The automation validates the approval result before allowing the request to progress through the appropriate workflow path.

Why

Network requests requiring authorization should not proceed as approved requests until the required approval has been completed.

How

The Flow Designer evaluates the result of the **Ask for Approval** action.

Outcome

The request follows the correct processing path based on the approval result, preventing an unapproved request from being treated as an approved request.

Access Control

What

Access to request data is controlled through ServiceNow's configured access-control mechanisms.

How

The custom Network Database table uses the ACL configuration established during the **Access Control** phase.

Appropriate permissions determine which users can access or modify the relevant records.

Outcome

Request data is protected from unauthorized access or modification.

QA Testing

What

The complete request lifecycle is tested to verify that data remains accurate throughout the process.

How

Testing covers:

1. Request submission.
2. Mandatory-field validation.
3. Variable population.
4. Network Database record creation.
5. Approval processing.
6. Approval/rejection branching.
7. Email notification.
8. Record status updates.

Test Matrix

Test Scenario	Expected Result
Required field left empty	Submission is prevented or validation is displayed

Test Scenario	Expected Result
Valid request submitted	Request is successfully created
User selected	Related requester information is populated where configured
Network Database record created	Catalog values are mapped correctly
Approval granted	Approved workflow path executes
Approval rejected	Rejected workflow path executes
Notification triggered	Appropriate email is generated
Record updated	Correct request status is stored

Data Integrity Controls Summary

Area	Actions Taken
Data Handling	Catalog variables mapped to structured custom table fields
Mandatory Validation	Required fields configured to prevent incomplete submissions
Auto-Population	User/context information populated automatically where configured
Access Control	ACL-based access control applied to the custom table
Approval Validation	Flow conditions evaluate approval state before progressing
QA Testing	Full-cycle testing of request, approval, notification, and record updates

Outcome

The Data Integrity implementation ensures that Network Requests are processed using complete, consistent, and validated information.

The combination of mandatory-field validation, auto-population, structured field mapping, approval-state checks, access control, and end-to-end testing provides data integrity throughout the request lifecycle.

Replicability Note

Data-integrity controls should be applied at the appropriate layer. Client-side controls such as UI Policies improve the user experience, while server-side validation and platform security provide stronger protection against invalid or unauthorized data. Approval-state conditions in Flow Designer ensure that subsequent processing is dependent on the actual approval outcome.